

9. Request and Response Validation

In API testing, **Request and Response Validation** is the most critical part because it ensures that every API is behaving exactly as expected according to business requirements and technical specifications.

In this project using Fake Store API, we validate every endpoint (Products, Cart, Users, Categories, Authentication) using structured assertions in Postman.

9.1 Status Code Validation

◆ What is it?

Status code validation means verifying the **HTTP response code** returned by the API.

Every API request returns a status code that tells whether the request is:

- Successful
 - Failed
 - Unauthorized
 - Not found, etc.
-

◆ Common Status Codes used in this project

Status Code	Meaning	Example
200	OK (Success)	GET Products
201	Created	Add Product / Add Cart
400	Bad Request	Missing required fields
401	Unauthorized	Invalid token
404	Not Found	Wrong Product ID
500	Server Error	Internal issue

◆ Postman Validation Script Example

```
pm.test("Status code is 200", function () {  
    pm.response.to.have.status(200);  
});
```

◆ Why it is important?

- Ensures API is responding correctly
 - Helps identify broken endpoints
 - Confirms correct backend processing
-

9.2 Response Body Validation

◆ What is it?

Response body validation means checking the **actual data returned from API**.

We verify:

- Required fields exist
 - Data is correct
 - Structure is proper (JSON format)
-

◆ Example (Get Product API Response)

```
{
  "id": 1,
  "title": "Fjallraven Backpack",
  "price": 109.95,
  "category": "men's clothing"
}
```

◆ Validation Points:

- `id` should not be null
 - `title` should be string
 - `price` should be number
 - `category` should match expected value
-

◆ Postman Script Example

```
pm.test("Validate product title is present", function () {
  var jsonData = pm.response.json();
  pm.expect(jsonData.title).to.not.be.null;
});
```

◆ Why important?

- Ensures correct data is returned
 - Detects missing or incorrect fields
 - Validates business logic
-

9.3 Response Time Validation

◆ What is it?

It checks how fast an API responds.

Slow APIs = bad user experience.

◆ Standard benchmark:

- < 2000 ms → Good
 - 3000 ms → Performance issue
-

◆ Postman Script Example

```
pm.test("Response time is less than 2000ms", function () {  
    pm.expect(pm.response.responseTime).to.be.below(2000);  
});
```

◆ Why important?

- Ensures performance quality
 - Detects slow APIs
 - Helps in load testing decisions
-

9.4 Environment Variable Validation

◆ What is it?

Environment variables are **dynamic values stored in Postman** to avoid hardcoding.

Examples:

- baseUrl
 - authToken
 - productId
-

◆ Example:

Instead of:

```
https://fakestoreapi.com/products/1
```

We use:

```
{{baseUrl}}/products/{{productId}}
```

◆ Validation Example

```
pm.test("Check base URL is set", function () {  
    pm.expect(pm.environment.get("baseUrl")).to.not.be.undefined;  
});
```

◆ Token Storage Example

```
var jsonData = pm.response.json();  
pm.environment.set("authToken", jsonData.token);
```

◆ Why important?

- Makes tests reusable
 - Supports multiple environments (QA, Dev, Prod)
 - Enables automation (Newman, CI/CD)
-

9.5 JSON Response Verification

◆ What is it?

This ensures that the API response is in valid JSON format and follows correct structure (schema validation).

◆ What we validate:

- Data types (string, number, boolean)
 - Required keys exist
 - No invalid/null structure
 - Proper nesting of objects
-

◆ Example Response:

```
{
  "id": 1,
  "title": "Shirt",
  "price": 25.5
}
```

◆ Postman Script Example

```
pm.test("Response is valid JSON", function () {
  pm.response.to.be.json;
});
```

◆ Data Type Validation Example

```
pm.test("Price should be number", function () {
  var jsonData = pm.response.json();
  pm.expect(jsonData.price).to.be.a('number');
});
```

◆ Why important?

- Ensures API contract is correct
 - Prevents frontend integration issues
 - Detects backend data issues early
-

✓ Final Summary (Interview Answer Style)

In this project, I implemented **Request and Response Validation** in Postman to ensure API quality and correctness. I validated status codes for correct HTTP responses, verified response body data for accuracy, checked response time for performance, used environment variables for dynamic testing, and ensured JSON response structure and data types were correct. This helped ensure that all APIs in the Fake Store system were stable, reliable, and ready for real-world usage.
